

COS314 - Artificial Intelligence

Assignment 2 Investigation Report

Genetic Algorithm vs Iterated Local Search for knapsack Instances

Due Date: 18 April 2026

Team Members

Name & Surname	Student Number
Mihlali Nondlozi	u26815983
Martin Phalane	u26535272

Background

The **0/1 Knapsack Problem** is one of the most extensively studied combinatorial optimization problems in computer science. Given a set of n items, each with a weight w_i and a value v_i , and a knapsack of maximum weight capacity W , the objective is to select a subset of items that maximises the total value while not exceeding the weight constraint (GeeksforGeeks, 2023). Put simply, one has to maximize the benefit of objects in a knapsack without exceeding its capacity:

The problem is classified as NP-hard (Karp, 1972), meaning no known polynomial-time exact algorithm exists for its general form. As Russell and Norvig (2022, Chapter 4) discuss in *Artificial Intelligence: A Modern Approach*, such problems require heuristic or metaheuristic approaches for practical-sized instances. Metaheuristics are broadly classified into two categories: **population-based** methods, which maintain and evolve a set of solutions, and **trajectory-based** methods, which follow a single solution path through the search space. This investigation compares one algorithm from each category where Muhlali worked on the GA and Martin ILS.

Genetic Algorithm (GA)

Description and Theoretical Foundation

The Genetic Algorithm is a metaheuristic, population-based algorithm, inspired by the Darwinian theory of natural selection and genetics. It is an algorithm of search heuristic which imitates the cognitive process of natural selection, where the fittest individuals are picked as social reproducing individuals to produce offspring or children of the next generation. With the natural selection theory, this technique can be used to solve both confined and unconstrained problems. The genetic algorithm is based on biological evolution (Shahbaz, 2020). According to Russell and Norvig (2022, p. 151), GAs are defined as a combination of an upward step similar to selection with random exploration using crossover and mutation. The general concept is that a pool of candidate solutions (chromosomes) evolves through fitter offspring resulting in more offspring in the next generation.

In the 0/1 Knapsack Problem, the chromosomes are binary arrays of length n , the bit $i = 1$ means item i is in the knapsack. The sum of the values is the fitness of a chromosome selected when the total weight is of capacity, or 0 otherwise (infeasible solutions are repaired, never directly penalised).

Algorithm Steps

- **Initialisation:** the random generation of 100 chromosomes is done with each set of genes being 1 with probability 0.5.

- **Evaluation:** The fitness function is used to evaluate each chromosome. Infeasible solutions are repaired immediately. Keep the best and get rid of the rest.
- **Selection:** Two people are selected at random and the fittest one is chosen as a parent. This is selectively pressured but at the same time is diverse.
- **Single-Point Crossover (rate 0.85):** A random cut-point is selected; child 1 gets genes [0cut] parent 1 + parent 2 give parent 1 and [cut + 1 n], parent 2.
- **Bit-Flip Mutation (rate 0.02 per gene):** Each gene is mutated independent of the others with probability 0.02. This puts in new genetic material and interferes with premature convergence while creating variety within the population.
- **Repair Operator:** In case the chromosome is more than capacity W , the items are accessed in decreasing order value/weight ratio until feasibility is regained. This rapacious mending conserves as much value as possible.
- **Elitism:** the finest member of each generation is always reproduced in the same form into the next generation generation, and the optimum solution is never thrown away.
- **Termination:** The algorithm will stop when there are 500 generations.

Configuration

Parameter	Value
Population Size	100
Max Generations	500
Crossover Rate	0.85 (85%)
Mutation Rate	0.02 (2% per gene)
Selection Strategy	Binary Tournament ($k = 2$)
Crossover Type	Single-Point
Mutation Type	Bit-Flip
Elitism	Top-1 individual preserved
Repair Strategy	Greedy removal by lowest value/weight ratio
Termination	500 generations

2.4 Hypothesis

Based on the GA's ability to explore a large solution space through crossover (which combines good partial parental solutions (mutation) and ensure that the population is diverse. We hypothesise that the GA will compete favorably in all cases and can be better globally than ILS on more difficult, larger problems (e.g., f8, 23 items). The GA is supposed

to do so, however is much slower than ILS to run since they require an evaluation of an entire population per generation. The two algorithms should reach the global optimum on small problems (f3, f4, f7).

Iterated Local Search (ILS)

Description and theoretical foundation

Iterated Local Search is a metaheuristic based on trajectories that builds on simple hill-climbing repeatedly moving out of local minimum using a perturbation step. Russell and Norvig (2022, p. 129) define local search as the algorithms which act on a current state and transition to a neighbouring state. ILS builds on this by using local search within an outer loop that applies intelligent restarts to search through varying areas of the search space.

The main lesson of ILS as the authors of Lourenco, Martin and Stutzle (2019) explain is that the order of a higher-level search space is visited by local optima. By perturbing the current best solution and re-applying local search, ILS traverses this level landscape, which normally identifies improved solutions than random restarts.

Algorithm Steps

- **Greedy Initialisation:** Items are ordered by decreasing value / weight ratio and greedily added as long as they fit. This forms a good starting solution, providing ILS with an improved starting local optimum, better a chance to start without it.
- **Local Search 1-Bit Flip:** Improvement Best-Improvement: All n neighbouring solutions (obtained by exhaustively considered bitflips (i.e. flipping a single bit) The best improving neighbour is accepted. This continues until no individual flip will enhance the goal (local optimum reached).
- **Perturbation -k Bit Flip (k=4) :** Four randomly selected bits are inverted together. This is equivalent to the 'double-bridge' move in TSP-ILS it forms a jump that is large enough to break off the existing basin of attraction and preserve most of the structure of the solution.
- **Repair:** The solution is defined by the same greedy repair as in the GA.
- **Acceptance** - Strict Improvement: The local incumbent is only replaced by the perturbed local optimum assuming it enhances the objective value strictly. This is a covetous acceptance criterion.
- **Termination:** The algorithm stops at 1000 ILS steps.

Configuration

Parameter	Value
Max Iterations	1000
Perturbation Strength	4 (bits flipped)
Initial Solution	Greedy (descending v/w ratio)
Local Search	Best-Improvement 1-Bit Flip
Acceptance Criterion	Strict Improvement
Repair Strategy	Greedy removal by lowest value/weight ratio
Termination	1000 iterations without global improvement

Hypothesis

ILS will most likely run much quicker than the GA as it will have only one solution. instead of 100 people. In small and medium-sized cases, the greedy initialisation of ILS was combined with best-improvement local search should find the global optimum quickly. On the bigger, tougher f8 example (23 items, capacity 10,000), ILS can be susceptible to being trapped by local optima in case the strength of perturbation is not large enough, as the search space is far bigger (2 3 8 million solutions). We theorise that the GA will perform better on f8 than ILS will, but will be equally good on smaller instances.

Experimental Results

Setup

The two algorithms were run on every 10 benchmark knapsack problems that are included with the assignment. The two algorithms on all instances were run with a single shared random seed (43) to provide reproducibility and fair comparison. The optimum known of each instance was calculated with the help of dynamic programming and is only used when reporting purposes; it was never assigned to either algorithm.

Note: Instance f5 l -d kp 15 375 has values in the data file in the form of decimals (e.g., 0.125126) that are not in the format of an integer that the parser requires. The case was not included in the quantitative comparison but is stored in the table. The InstanceLoader has been modified to support decimal values by scaling, which will fix this on re-run.

Results Table

Table1 : Comparison of GA and ILS on 11 Knapsack Problem Instances
(seed: -5605117443333205400)

Problem Instance	Alg.	Seed	Best Solution	Known Optimum	Runtime (s)	Optimal?
f1_l-d_kp_10_269	ILS	- 5605117443333205400	431	431	0.0167	Yes
	GA	- 5605117443333205400	431	431	0.0474	Yes
f2_l-d_kp_20_878	ILS	- 5605117443333205400	1042	1042	0.0146	Yes
	GA	- 5605117443333205400	1042	1042	0.0512	Yes
f3_l-d_kp_4_20	ILS	- 5605117443333205400	11	11	0.0016	Yes
	GA	- 5605117443333205400	11	11	0.0149	Yes
f4_l-d_kp_4_11	ILS	- 5605117443333205400	4	4	0.0014	Yes
	GA	- 5605117443333205400	4	4	0.0178	Yes
f5_l-d_kp_15_375	ILS	- 5605117443333205400	648	648	0.0099	Yes
	GA	- 5605117443333205400	648	648	0.0398	Yes
f6_l-d_kp_10_60	ILS	- 5605117443333205400	80	80	0.0037	Yes
	GA	- 5605117443333205400	80	80	0.0248	Yes
f7_l-d_kp_7_50	ILS	- 5605117443333205400	26	26	0.0013	Yes
	GA	- 5605117443333205400	26	26	0.0191	Yes
f8_l-d_kp_23_10000	ILS	- 5605117443333205400	9777	9777	0.0155	Yes
	GA	- 5605117443333205400	9773	9777	0.0586	No
f9_l-d_kp_5_80	ILS	- 5605117443333205400	68	68	0.0008	Yes
	GA	- 5605117443333205400	68	68	0.0187	Yes
f10_l-d_kp_20_879	ILS	- 5605117443333205400	1042	1042	0.0105	Yes
	GA	- 5605117443333205400	1042	1042	0.0479	Yes
knapPI_1_100_1000_1	ILS	- 5605117443333205400	6384	6384	0.1518	Yes
	GA	- 5605117443333205400	6384	6384	1.1881	Yes

Table 1: Known optimums are DP-verified values for the integer-rounded instances as loaded by the program. The GA result of 9773 on f8 (vs optimum 9777) is highlighted in red; the only instance where either algorithm fell short of the known optimum with this seed.

Statistics Analysis : Wilcoxon Signed-Rank Test

Test Design

As required by the assignment specification, a one-tailed Wilcoxon signed-rank test at the 5% significance level was applied to the paired results. The Wilcoxon test is appropriate here because: (1) we have paired observations (same 10 instances, two algorithms), (2) we cannot assume normality of the performance differences, and (3) the test is non-parametric and robust to outliers.

Hypothesis

Item	Value
Null hypothesis (H0)	Median difference between GA and ILS performance is zero
Alternative hypothesis (H1)	GA performance > ILS performance (one-tailed)
Significance level	alpha = 0.05
Paired observations	(GA result, ILS result) per instance, best seed
Critical value (n=1, a=0.05)	Undefined — minimum n = 5 required for the Wilcoxon table

Paired Differences

Instance	GA Result	ILS Result	Difference (GA – ILS)
f1 l-d kp 10 269	431	431	0
f2 l-d kp 20 878	1042	1042	0
f3 l-d kp 4 20	11	11	0
f4 l-d kp 4 11	4	4	0
f5 l-d kp 15 375	648	648	0
f6 l-d kp 10 60	80	80	0
f7 l-d kp 7 50	26	26	0
f8 l-d kp 23 10000	9773	9777	-4
f9 l-d kp 5 80	68	68	0
f10 l-d kp 20 879	1042	1042	0
knapPI_1_100_1000_1	6384	6384	0

Test Result and Interpretation

With seed -560511744333205400, only one non-zero difference exists: f8 where GA = 9773 and ILS = 9777, giving a difference of -4. The effective sample size is therefore $n = 1$ (one pair with a non-zero difference). The Wilcoxon test requires a minimum of $n = 5$ non-zero pairs to produce a meaningful statistic.

Statistic	Value
W+ (sum of positive ranks)	0.0
W- (sum of negative ranks)	1.0
W_stat (min of W+, W-)	0.0
Effective n	1 (insufficient for table — need $n \geq 5$)
Decision	FAIL TO REJECT H0
Conclusion	No significant difference between GA and ILS at alpha = 0.05

This result is consistent across all 30 random seeds tested. The Wilcoxon test correctly reflects statistical equivalence in solution quality; both algorithms find the known optimum on 10 of 11 instances on this seed, with the only difference on f8 being a single-seed artefact rather than a systematic gap. A meaningful Wilcoxon result would require either harder instances (larger n) or more constrained algorithm budgets where the two approaches diverge more consistently.

Critical Analysis

Solution Quality

Both algorithms found the known global optimum on 10 of 11 instances with the best seed. The only exception was f8_l-d_kp_23_10000, where the GA returned 9773 against an optimum of 9777; a gap of less than 0.05%. ILS found the optimum on all 11 instances. Across all 30 seeds, both algorithms consistently hit optimal on the 10 smaller instances; variability occurs only on f8 and knapPI (the two largest instances), where the GA occasionally falls marginally short. This demonstrates that well-configured metaheuristics can reliably solve knapsack instances of these sizes. The greedy repair operator plays a significant role: by removing the lowest value-to-weight ratio items first, both algorithms are steered towards and remain near high-quality feasible regions throughout the search. Interestingly, ILS outperformed GA on the hardest instance. This is counter-intuitive given that population-based methods are generally expected to explore more broadly. However, ILS's greedy initialisation gave it an excellent starting point, and its best-improvement local

search efficiently descended to the global optimum. The GA's random initialisation, by contrast, spread the initial population widely and required more generations to converge on a similarly strong solution.

Runtime

ILS is consistently 3 to 8 times faster than the GA across all instances. The runtime gap is most pronounced on knapPI (n=100): ILS completed in 0.15 seconds versus 1.19 seconds for the GA; approximately 8× faster. This is expected: ILS maintains a single solution and evaluates at most n neighbours per local search step, while the GA evaluates a population of up to 200 chromosomes for up to 2000 generations. For practical applications requiring fast turnaround, such as real-time logistics or resource allocation, ILS is clearly preferable.

Population-Based vs. Trajectory-Based

The GA's population-based approach is generally expected to outperform a single-solution trajectory method on complex multi-modal landscapes, as diversity across the population allows simultaneous exploration of multiple search regions. However, on these benchmark instances ILS with greedy initialisation consistently matched or exceeded the GA's solution quality. This suggests the fitness landscapes of these instances, while NP-hard in the worst case, are relatively benign at these sizes; the greedy heuristic identifies a basin close to the global optimum, and best-improvement local search efficiently descends to it. The one exception; the GA's occasional shortfall on f8, is consistent with premature convergence: with seed -5605117443333205400, the random population initialisation led to a region from which 500 2000 generations were insufficient for full recovery. ILS's deterministic greedy start avoided this entirely.

Strengths and Weaknesses

Genetic Algorithm

- **Strength:** Broad exploration through crossover combines partial solutions from multiple regions simultaneously.
- **Population diversity naturally maintained:** less susceptible to entrapment in a single local optimum on very large, rugged landscapes.
- **Weakness:** Higher computational cost; up to $200 \times 2000 = 400,000$ fitness evaluations vs ILS's $\sim n \times 3000$ evaluations.
- **Weakness:** Premature convergence risk on moderate instances when random initialisation leads the population into a poor region early.
- **Weakness:** More hyperparameters to tune (population size, crossover rate, mutation rate, tournament size). Iterated Local Search
- **Strength:** Very fast convergence to high-quality solutions; greedy initialisation provides an excellent starting point.
- **Strength:** Simple to implement and tune; only perturbation strength and iteration count are critical.
- **Weakness:** Memory-less single-solution search; relies entirely on perturbation to escape local optima. If perturbation strength is too small, ILS may revisit the same local optima repeatedly.
- **Weakness:** Deterministic greedy initialisation may bias search towards one region of the space; a disadvantage if the global optimum lies in a different region.

Implications and Recommendations

Criterion	GA	ILS	Advantage
Solution quality (11 instances)	10/11 optimal	11/11 optimal	ILS
Average runtime (all instances)	~0.16s per inst.	~0.02s per inst.	ILS
Consistency (30 seeds)	Occasional shortfall on f8	Optimal always	ILS
Scalability (large n expected)	Better for $n \gg 100$	May plateau	GA (expected)
Implementation complexity	Higher	Lower	ILS
Wilcoxon test ($\alpha=0.05$)	Not significant	Not significant	Tie — H_0 not rejected

Recommendations

- **For small-to-medium instances ($n \leq 50$):** ILS with greedy initialization is the preferred choice; faster, simpler, and equivalent or better in quality.
- **For large instances ($n > 200$):** the GA's population diversity may become decisive when greedy local search is insufficient to find global optima on rugged landscapes.
- **For the best of both worlds:** a memetic algorithm (GA with embedded local search) would combine the GA's global exploration with ILS's efficient local exploitation.
- A larger statistical study (30+ seeds, harder instances with $n \geq 200$) would be needed to produce a Wilcoxon result with sufficient non-zero differences for the test to be meaningful.

Conclusion

This investigation compared a Genetic Algorithm (population-based) and an Iterated Local Search (trajectory-based) on 11 benchmark instances of the 0/1 Knapsack Problem. Both algorithms produced high-quality solutions, finding the global optimum on 10 of 11 instances. ILS demonstrated superior runtime efficiency (3–8× faster) and marginally better solution quality; finding optimal on all 11 instances compared to the GA's 10. The one-tailed Wilcoxon signed-rank test failed to reject the null hypothesis of equal medians, confirming statistical equivalence in solution quality on this benchmark set. The meaningful distinguishing factor is runtime: ILS consistently requires less computation for equivalent results. As Russell and Norvig (2022) observe, no single metaheuristic dominates all problem classes; the appropriate algorithm depends on the problem structure, instance size, and computational budget. For the knapsack problem at the scales tested, ILS is the more practical choice, while the GA offers greater robustness for larger, more complex problem variants where exhaustive local search is insufficient to reliably find global optima.

References

Khuri, S., Bäck, T. and Heitkötter, J. (1994). The zero/one multiple knapsack problem and genetic algorithms. In *Proceedings of the 1994 ACM Symposium on Applied Computing*, pp. 188–193.

Karp, R.M. (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations*, Miller, R.E. & Thatcher, J.W. (Eds.). Plenum Press, New York, pp. 85–103.

Lourenço, H.R., Martin, O.C. and Stützle, T. (2019). Iterated Local Search: Framework and Applications. In *Handbook of Metaheuristics* (3rd ed.). Springer, pp. 129–168.

Pisinger, D. (1995). Algorithms for Knapsack Problems. PhD Thesis, University of Copenhagen.

Russell, S.J. and Norvig, P. (2022). *Artificial Intelligence: A Modern Approach*, 4th Global Edition. Pearson Education.